

AWS — Scalable Storage & Monitoring (EBS + EFS + CloudWatch + SNS)

Project summary (goal)

Build a small but production-representative environment that demonstrates:

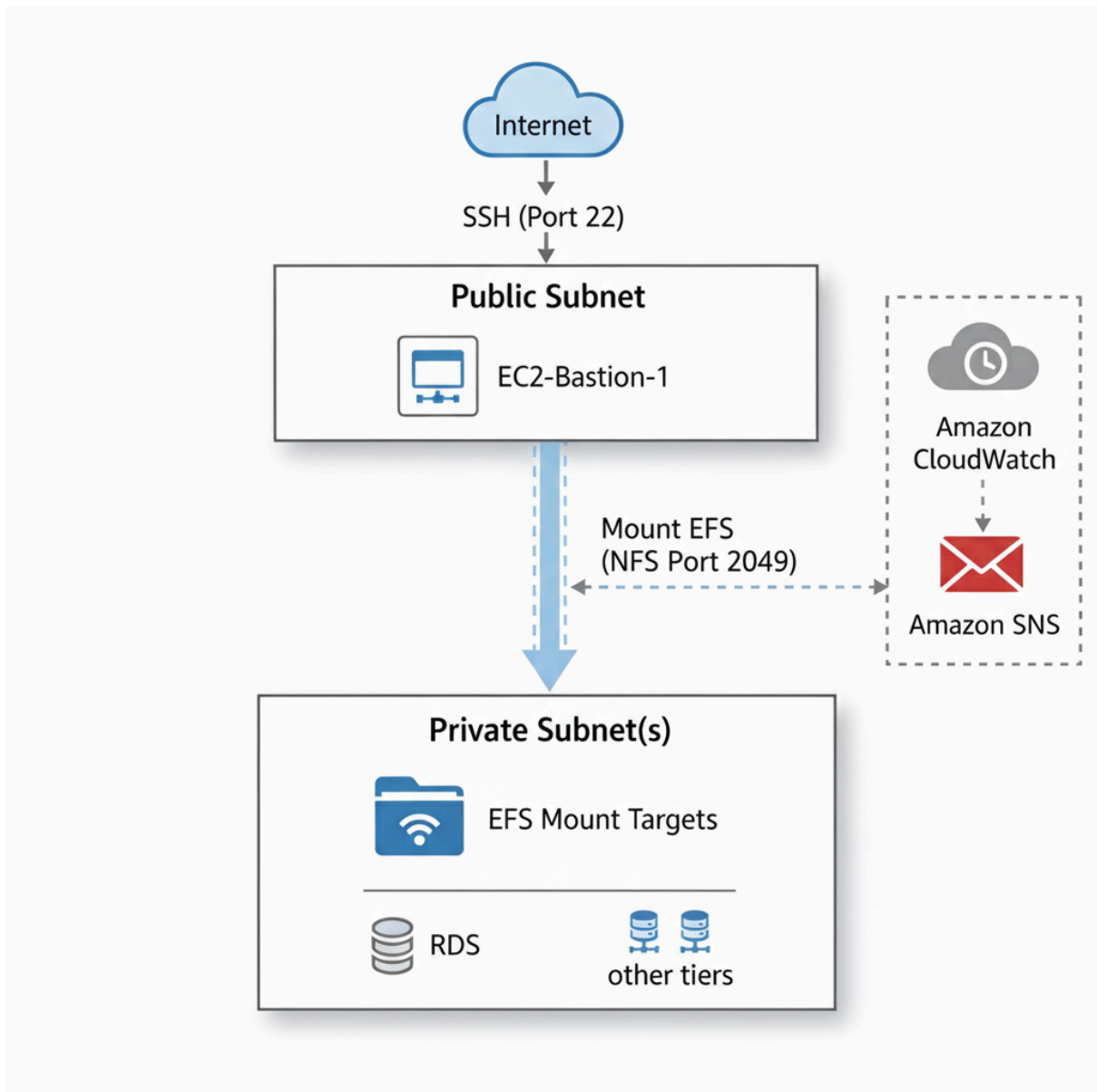
- Block storage lifecycle (EBS): attach, format, snapshot, resize.
- Shared file system (EFS): mount across multiple EC2s and verify file sharing.
- Observability (CloudWatch): metrics + logs via CloudWatch Agent.
- Alerting (SNS): CloudWatch alarms notify via SNS (email).
- Tests: EBS persistence, EFS replication to other instances, CloudWatch logs showing, SNS alert triggered.

Prerequisites

- An AWS account with sufficient rights (EC2, EBS, EFS, CloudWatch, SNS, IAM, VPC).
- AWS CLI configured on your machine (or use console for some steps).
- Region: choose one and stay consistent, e.g. `ap-south-1`.
- Work in a sandbox account to avoid charges.

Cost note: NAT Gateways and EFS and CloudWatch metrics have costs. Clean up when finished.

Architecture (simple)



EBS is attached to each EC2 (local block device). EFS is shared across EC2s using mount targets in the VPC subnets. CloudWatch agent on each EC2 pushes metrics/logs to CloudWatch. Alarms in CloudWatch publish to SNS.

Step-by-step build

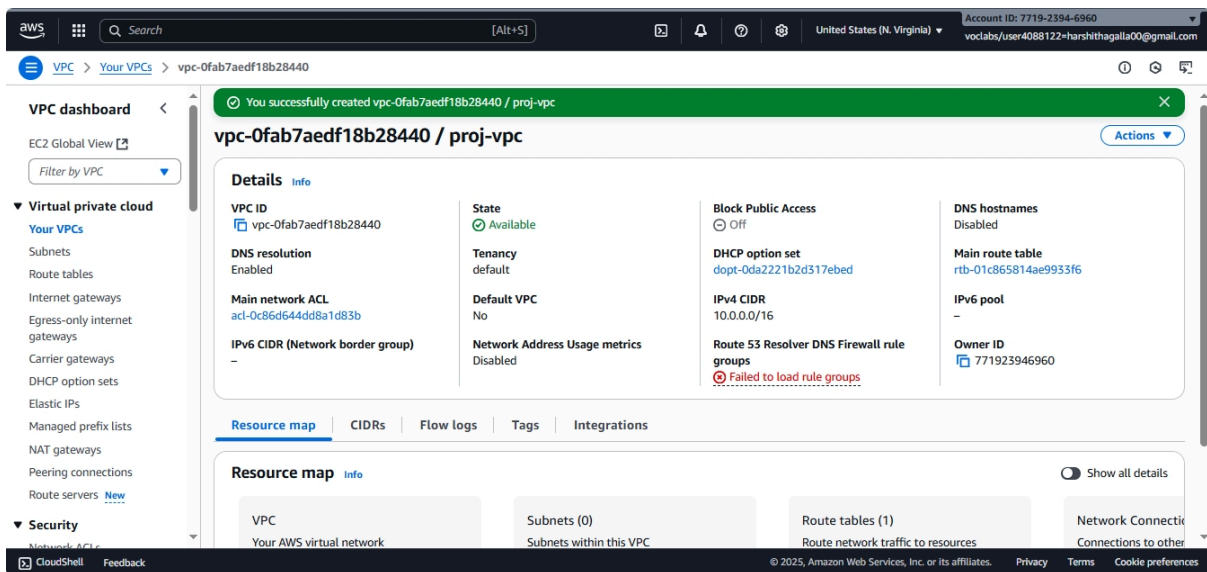
Follow steps in order. I give both Console guidance and CLI commands where appropriate.

Step 1: Create VPC & Subnets

Console Steps:

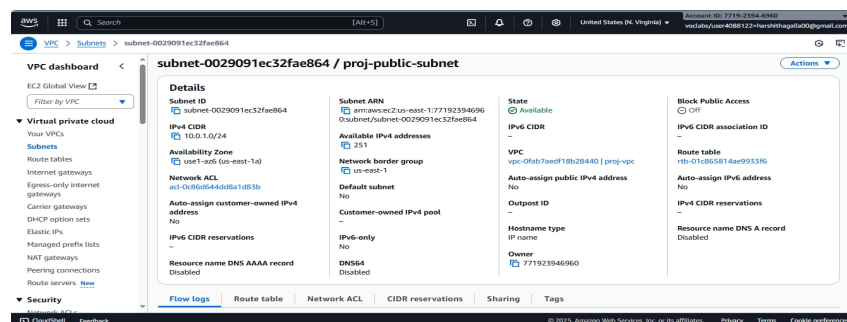
1. VPC → Your VPCs → Create VPC

- Name: proj-vpc
- IPv4 CIDR: 10.0.0.0/16
- Tenancy: Default → Create

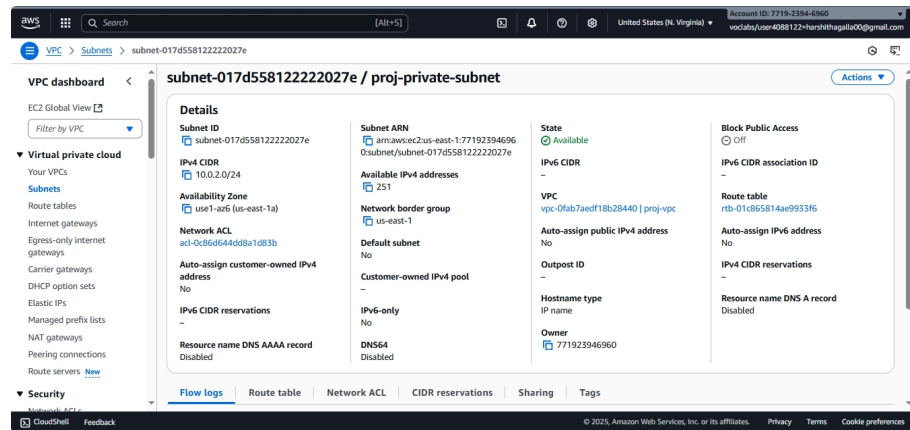


2. Create Subnets

- Public: 10.0.1.0/24 → AZ1 → Name: proj-public-subnet

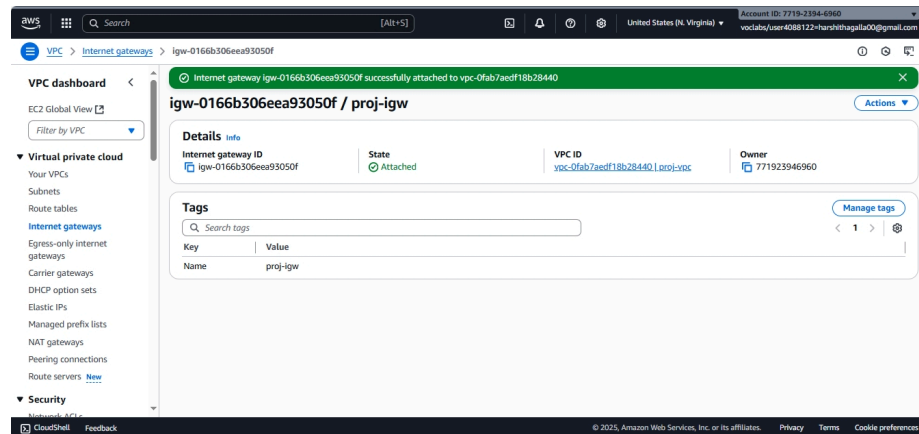


- Private 1: 10.0.2.0/24 → AZ1 → Name: proj-private-subnet



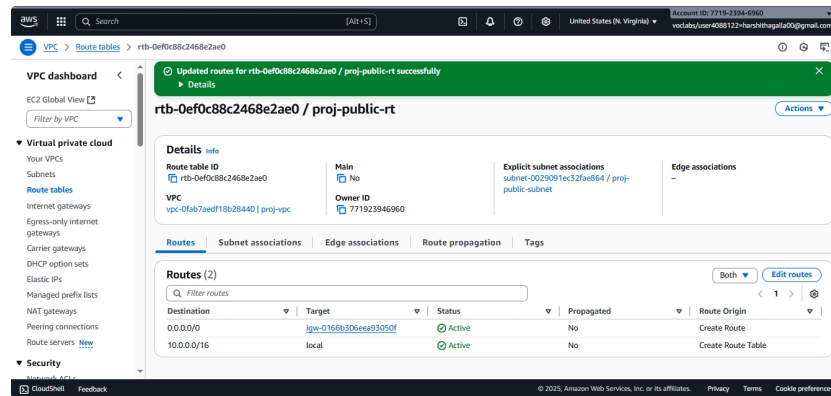
3. Internet Gateway (IGW)

- VPC → Internet Gateways → Create → Name: proj-igw → Attach to proj-vpc

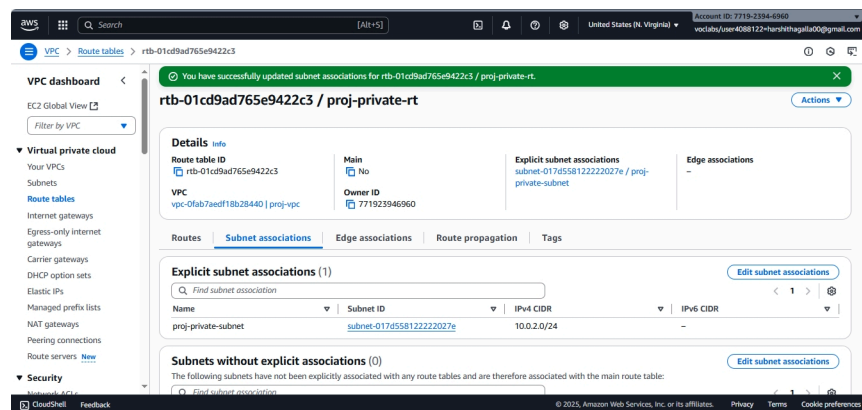


4. Route Table for Public Subnet

- Route Tables → Create → Name: proj-public-rt → Associate proj-public-subnet
- Edit Routes → Add 0.0.0.0/0 → Target: IGW → Save
- Associate public subnet



- Route Tables → Create → Name: proj-private-rt → Associate proj-public-subnet
- Associate private subnet



5. Optional NAT Gateway (for private subnet outbound access)

- NAT Gateways → Create → Public Subnet → Elastic IP → Name: proj-nat

CLI (optional):

```
aws ec2 create-vpc --cidr-block 10.0.0.0/16 --tag-specifications \
'ResourceType=vpc,Tags=[{Key=Name,Value=proj-vpc}]' --region
<REGION>
```

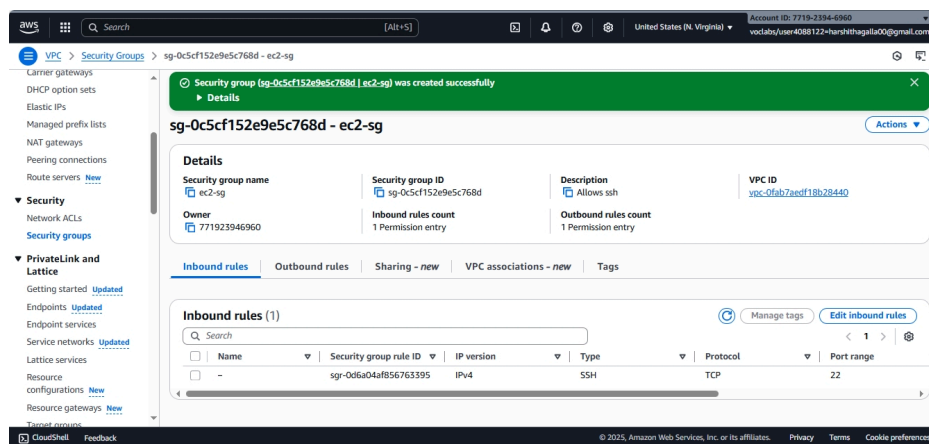
Step 2: Create Security Groups

Console Steps:

- VPC → Security Groups → Create → assign rules below

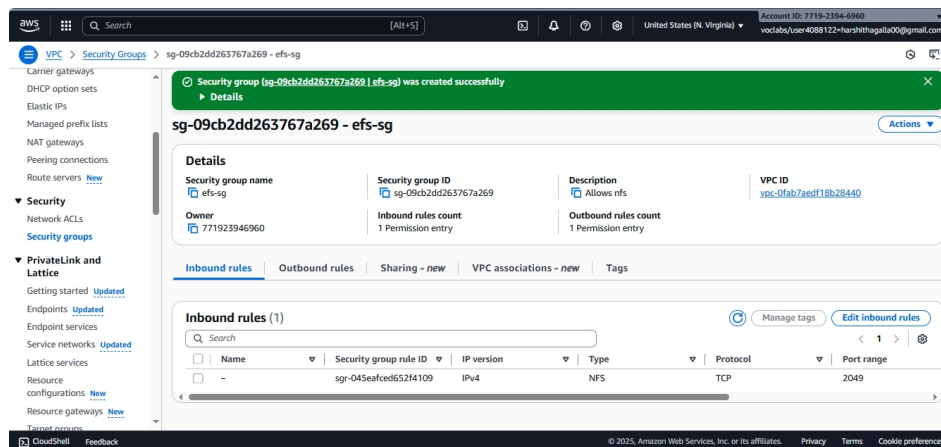
EC2 SG (ec2-sg):

- Inbound: SSH (22) from your IP
- Outbound: All traffic



EFS SG (efs-sg):

- Inbound: NFS (TCP 2049) from sg-ec2
- Outbound: All traffic



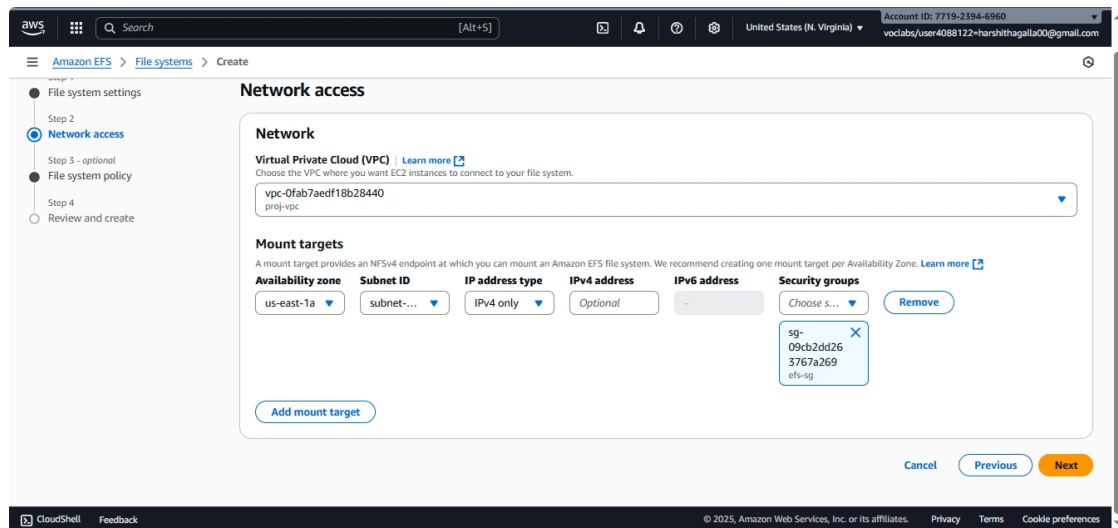
CLI example

```
aws ec2 create-security-group --group-name sg-ec2 --description "EC2 SG" --vpc-id <VPC_ID>
aws ec2 create-security-group --group-name sg-efs --description "EFS SG" --vpc-id <VPC_ID>
# allow SSH from your IP
aws ec2 authorize-security-group-ingress --group-id <SG_EC2_ID> --protocol tcp --port 22 --cidr <YOUR_IP>/32
# allow NFS from ec2 sg to efs sg
aws ec2 authorize-security-group-ingress --group-id <SG_EFS_ID> --protocol tcp --port 2049 --source-group <SG_EC2_ID>
```

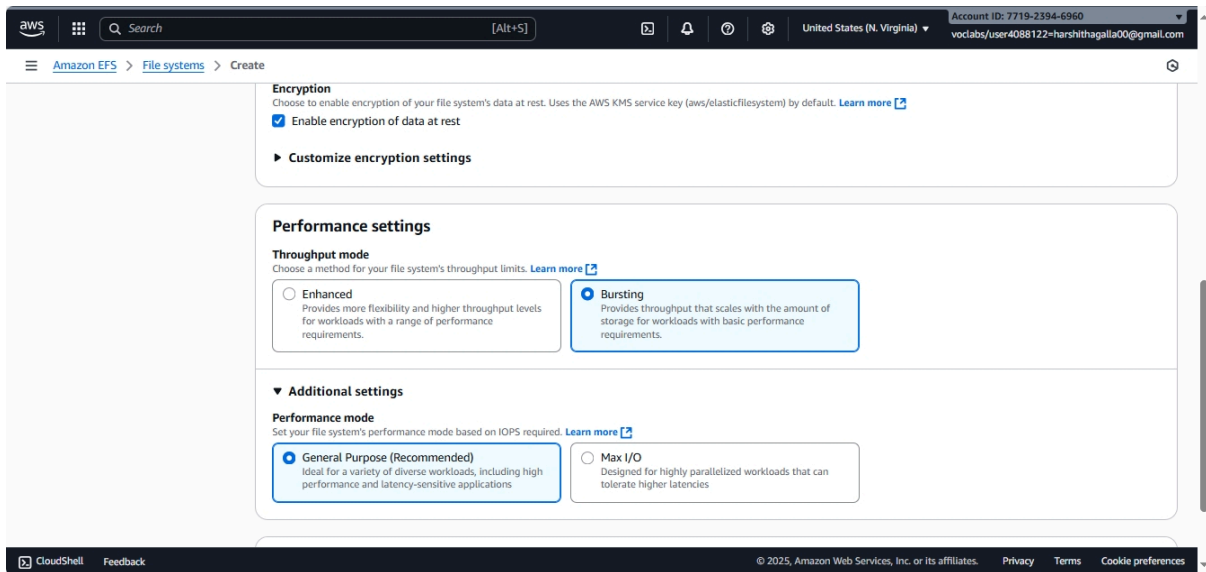
Step 3: Create EFS File System

Console

1. Go to EFS → Create file system.
2. Select VPC proj-vpc.
3. Create mount targets in private subnets and choose sg-efs for the mount targets.

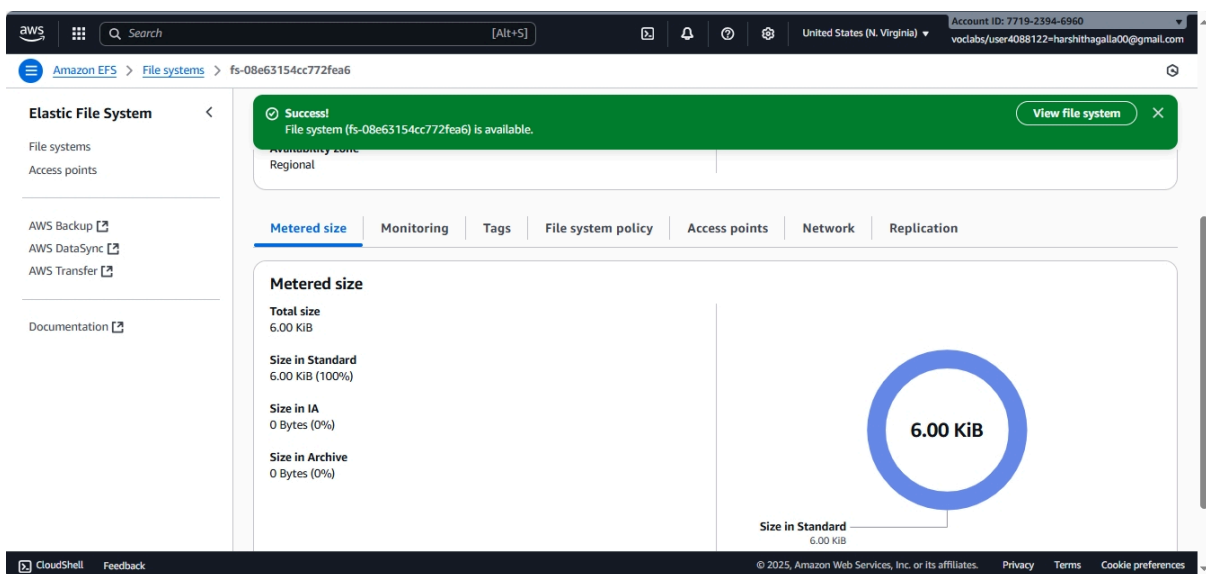


4. Performance mode: General Purpose; Throughput: Bursting (default) for dev.



5. Enable lifecycle policy if desired (e.g., move infrequent files to Infrequent Access).

6. Create.



CLI quick

```
aws efs create-file-system --performance-mode generalPurpose --tags
```

```
Key=Name,Value=proj-efs --region <REGION>
```

```
# create mount targets (example)
```

```
aws efs create-mount-target --file-system-id <FS_ID> --subnet-id <SUBNET_ID_A>
```

```
--security-groups <SG_EFS_ID>
```

```
aws efs create-mount-target --file-system-id <FS_ID> --subnet-id <SUBNET_ID_B>
```

```
--security-groups <SG_EFS_ID>
```


Record EFS FileSystemId (e.g. fs-xxxxxxx) and DNS name:
<fs-id>.efs.<region>.amazonaws.com.

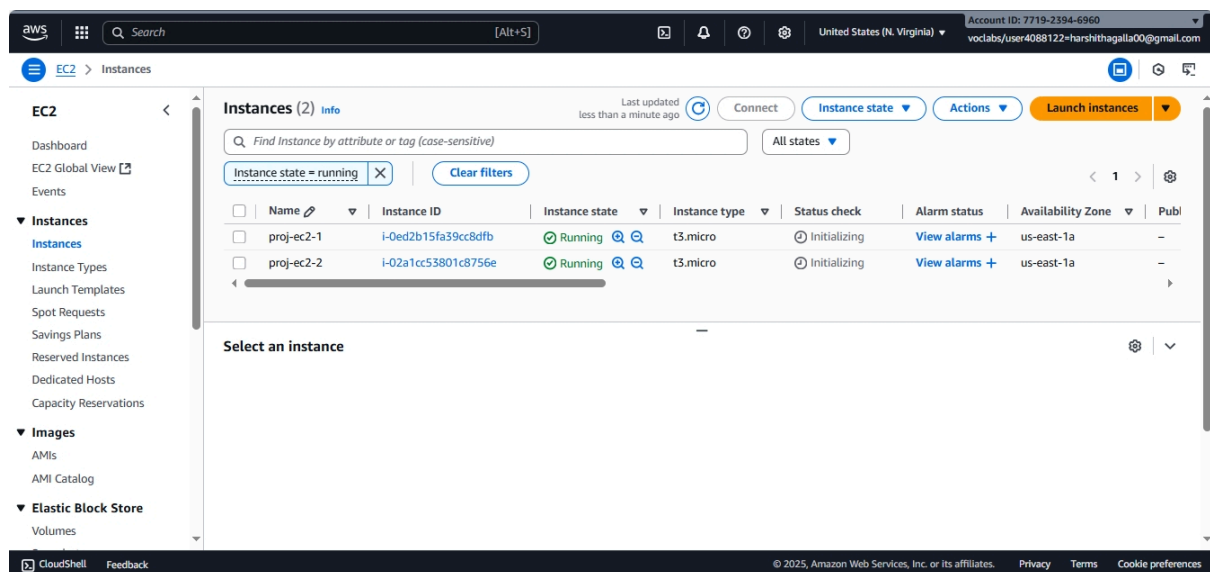
Step 4: Launch EC2 Instances

Launch two EC2 instances (Amazon Linux 2) in subnets (public or private with NAT). Attach sg-ec2.

Console Steps:

1. EC2 → Launch Instance

- Name: proj-ec2-1
- AMI: Amazon Linux
- Type: t3.micro
- Network: proj-vpc → Subnet: proj-public-subnet → Auto-assign public IP: Enable
- Security group: sg-ec2
- Key pair: choose existing or create → Launch
- Repeat for proj-ec2-2



CLI

```
aws ec2 run-instances --image-id <AMI_ID> --count 2 --instance-type t3.micro \
  --key-name <KEY> --subnet-id <PUBLIC_SUBNET_ID> --security-group-ids
  <SG_EC2_ID> \
  --associate-public-ip-address --tag-specifications
  'ResourceType=instance,Tags=[{Key=Name,Value=proj-ec2-1}]'
```

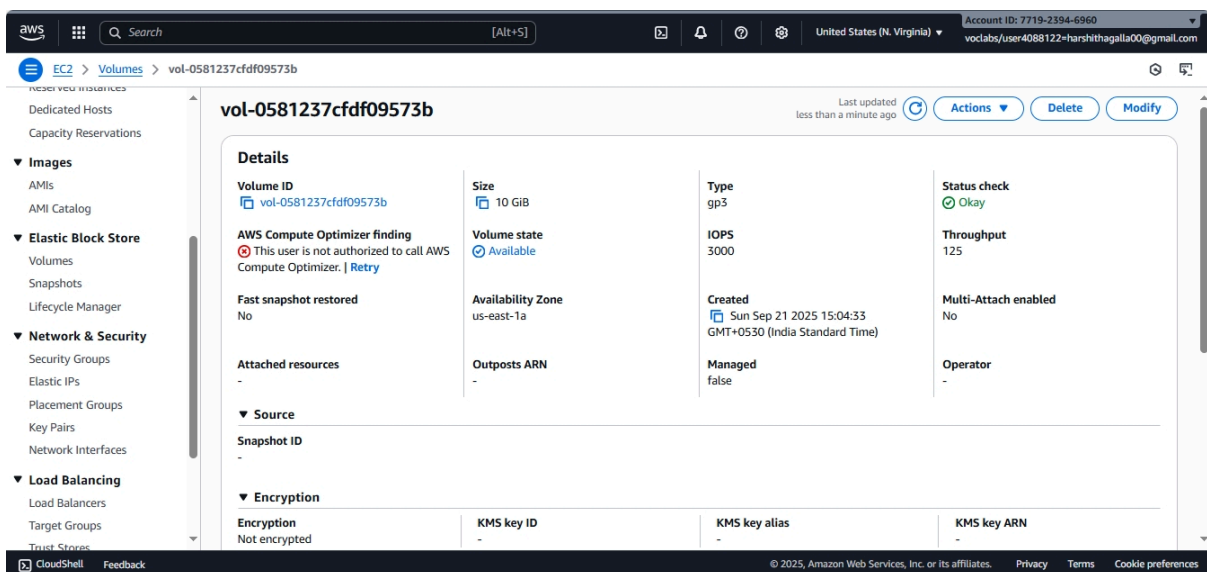
Note instance IDs and public IPs.

Step 5: Create and Attach EBS Volume

Create a 10 GiB gp3 EBS volume in same AZ as EC2 instance and attach.

Console Steps:

1. **EC2 → Elastic Block Store → Volumes → Create Volume**
 - Size: 10 GiB → Type: gp3 → AZ: same as EC2 → Name: proj-ec2-1



2. Attach volume → select instance → Device: /dev/xvdf

Attached resources

i-0ed2b15fa39cc8dfb (proj-ec2-1):
/dev/xvdf (attached)

CLI

```
# create volume in the same AZ as instance (e.g., ap-south-1a)
```

```
aws ec2 create-volume --availability-zone <AZ> --size 10 --volume-type gp3
--tag-specifications 'ResourceType=volume,Tags=[{Key=Name,Value=proj-eks-1}]' --region
<REGION>
```

```
# attach to instance
```

```
aws ec2 attach-volume --volume-id <VOLUME_ID> --instance-id <INSTANCE_ID> --device
/dev/xvdf
```

On EC2 (format + mount)

SSH to the instance:

```
# check device path; EBS may appear as /dev/nvme1n1 on Nitro instances
```

```
lsblk # assuming device is /dev/xvdf or /dev/nvme1n1
```

```
sudo file -s /dev/nvme1n1
```

```
#  
~\##### Amazon Linux 2023  
~~~\#####  
~~~\###|  
~~~\#/ https://aws.amazon.com/linux/amazon-linux-2023  
~~~~v~'-'->  
~~~~  
~~~~.-.  
~~~~/_m/'-/_
```

```
[ec2-user@ip-10-0-1-180 ~]$ lsblk  
NAME        MAJ:MIN RM SIZE RO TYPE MOUNTPOINTS  
nvme0n1      259:0    0   8G  0 disk  
└─nvme0n1p1  259:1    0   8G  0 part /  
└─nvme0n1p127 259:2    0  1M  0 part  
└─nvme0n1p128 259:3    0 10M  0 part /boot/efi  
nvme1n1      259:4    0 10G  0 disk  
[ec2-user@ip-10-0-1-180 ~]$ sudo file -s /dev/nvme1n1  
/dev/nvme1n1: data  
[ec2-user@ip-10-0-1-180 ~]$ ||
```

```
# format (xfs recommended on Amazon Linux)
```

```
sudo mkfs -t xfs /dev/nvme1n1
```

```
[ec2-user@ip-10-0-1-180 ~]$ sudo mkfs -t xfs /dev/nvme1n1
meta-data=/dev/nvme1n1      isize=512    agcount=16, agsize=163840 blks
       =                     sectsz=512    attr=2, projid32bit=1
       =                     crc=1        finobt=1, sparse=1, rmapbt=1
       =                     reflink=1    bigtime=1 inobtcount=1 nrext64=1
       =                     exchange=0
data      =                     bsize=4096    blocks=2621440, imaxpct=25
       =                     sunit=1      swidth=1 blks
naming    =version 2         bsize=4096    ascii-ci=0, ftype=1, parent=0
log       =internal log     bsize=4096    blocks=16384, version=2
       =                     sectsz=512    sunit=1 blks, lazy-count=1
realtime  =none             extsz=4096    blocks=0, rtextents=0
[ec2-user@ip-10-0-1-180 ~]$ ||
```

create mount point and mount

```
sudo mkdir -p /data/ebs
```

```
sudo mount /dev/nvme1n1 /data/ebs
```

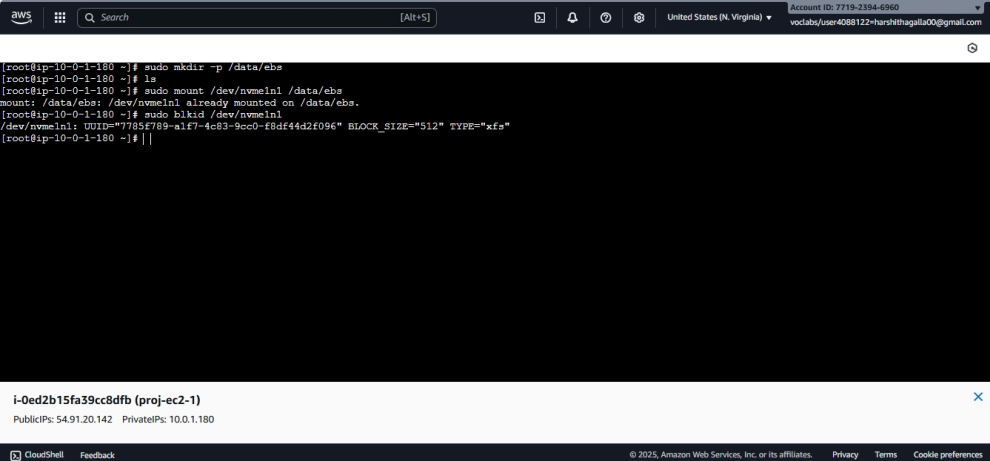
set fstab for persistence (use UUID)

```
sudo blkid /dev/nvme1n1(/dev/nvme1n1: UUID="e2ed8fc4-ef85-4ffa-8d3a-80626204e30f"
```

```
BLOCK_SIZE="512" TYPE="xfs")
```

edit /etc/fstab and add:

```
# UUID=<UUID> /data/ebs xfs defaults,nofail 0 2
```



```
aws
[Alt+S]
United States (N. Virginia)
Account ID: 7719-2394-6960
vodlabs/user4088122@harshthagalla00@gmail.com

[root@ip-10-0-1-180 ~]# sudo mkdir -p /data/ebs
[root@ip-10-0-1-180 ~]# ls
[root@ip-10-0-1-180 ~]# sudo mount /dev/nvme1n1 /data/ebs
mount: /data/ebs: /dev/nvme1n1 already mounted on /data/ebs.
[root@ip-10-0-1-180 ~]# sudo blkid /dev/nvme1n1
/dev/nvme1n1: UUID="7785f789-a1f7-4c83-9cc0-f8df44d2f096" BLOCK_SIZE="512" TYPE="xfs"
[root@ip-10-0-1-180 ~]# ||

i-0ed2b15fa39cc8dfb (proj-ec2-1)
PublicIPs: 54.91.20.142 PrivateIPs: 10.0.1.180

CloudShell Feedback
© 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences
```

Resize example

To increase size from 10 → 20 GiB (CLI + expand on instance):

```
aws ec2 modify-volume --volume-id <VOLUME_ID> --size 20
```

on instance:

```
sudo yum install -y cloud-utils-growpart # if needed
```

```
sudo growpart /dev/xvdf 1 # only if partition exists
```

for xfs

```
sudo xfs_growfs /data/ebs
```

for ext4

```
sudo resize2fs /dev/xvdf
```

Note: device names differ on Nitro instances (nvme). Use `lsblk` to confirm.

Snapshot

```
aws ec2 create-snapshot --volume-id <VOLUME_ID> --description "snapshot before upgrade"
```

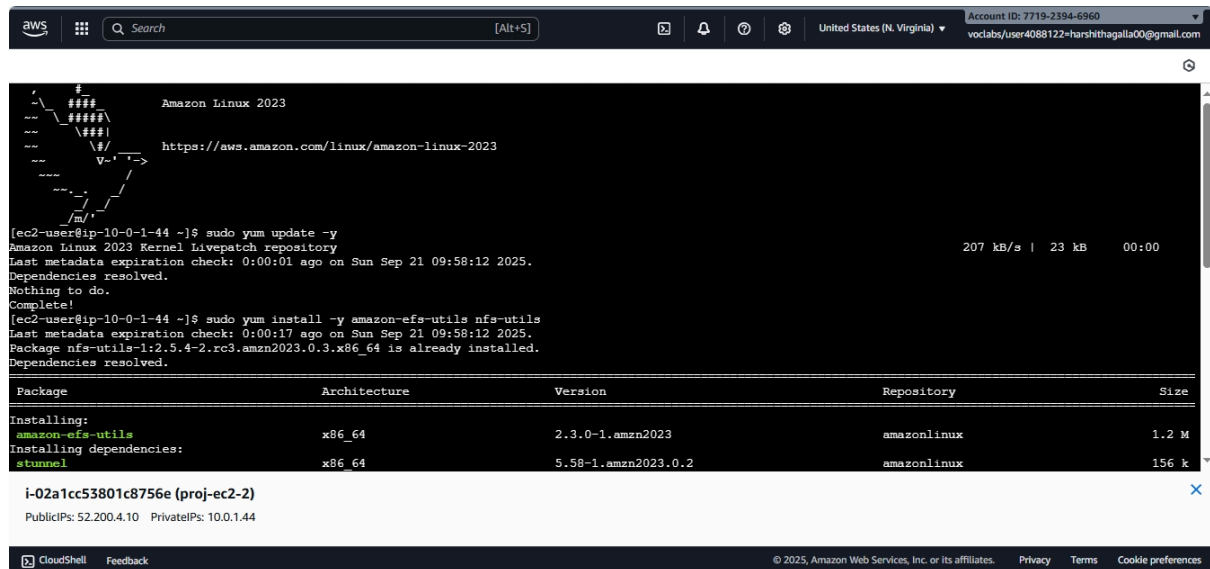
Step 6: Mount EFS on both EC2 instances

Install EFS utils and mount helper.

On each EC2 (Amazon Linux 2)

```
sudo yum update -y
```

```
sudo yum install -y amazon-efs-utils nfs-utils
```



The screenshot shows the AWS CloudShell interface with the Amazon Linux 2023 logo and a terminal window. The terminal output shows the following commands and results:

```
[ec2-user@ip-10-0-1-44 ~]$ sudo yum update -y
Amazon Linux 2023 Kernel Livepatch repository
Last metadata expiration check: 0:00:01 ago on Sun Sep 21 09:58:12 2025.
Dependencies resolved.
Nothing to do.
Complete!

[ec2-user@ip-10-0-1-44 ~]$ sudo yum install -y amazon-efs-utils nfs-utils
Last metadata expiration check: 0:00:17 ago on Sun Sep 21 09:58:12 2025.
Package nfs-utils-1:2.5.4-2.rc3.amzn2023.0.3.x86_64 is already installed.
Dependencies resolved.
```

Package	Architecture	Version	Repository	Size
Installing: amazon-efs-utils	x86_64	2.3.0-1.amzn2023	amazonlinux	1.2 M
Installing dependencies: stunnel	x86_64	5.58-1.amzn2023.0.2	amazonlinux	156 k

i-02a1cc53801c8756e (proj-ec2-2)
PublicIPs: 52.200.4.10 PrivateIPs: 10.0.1.44

```
# create mount point
```

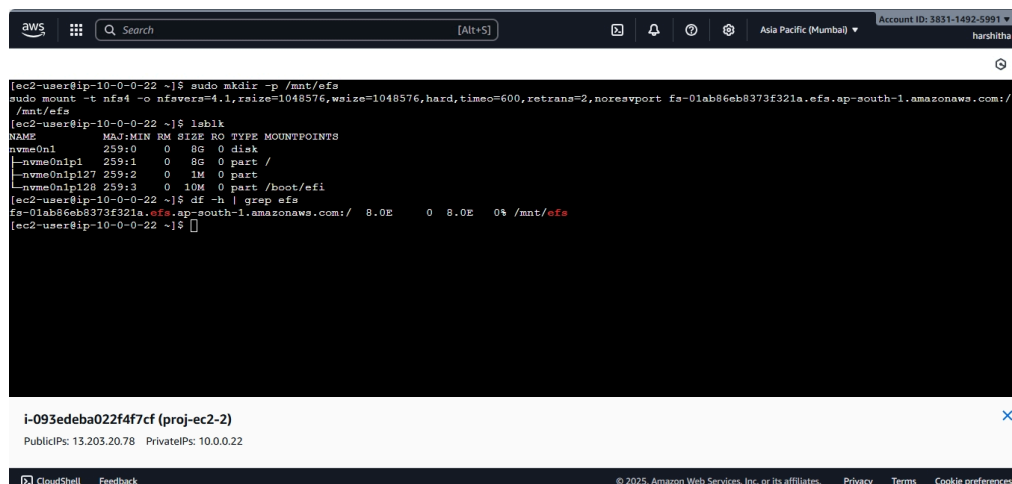
```
sudo mkdir -p /mnt/efs
```

```
# mount using mount helper
```

```
sudo mount -t efs -o tls <FS_ID>:/ /mnt/efs (sudo mount -t efs -o tls fs-08e63154cc772fea6:/  
efs)
```

```
# or DNS name:
```

```
sudo mount -t nfs4 -o nfsvers=4.1,rsize=1048576,wsiz=1048576,hard,timeo=600,retrans=2  
<FS_ID>.efs.<REGION>.amazonaws.com:/ /mnt/efs
```



The screenshot shows the AWS CloudShell interface with the Asia Pacific (Mumbai) region selected. The terminal output shows the following commands and results:

```
[ec2-user@ip-10-0-0-22 ~]$ sudo mkdir -p /mnt/efs
sudo mount -t nfs4 -o nfsvers=4.1,rsize=1048576,wsiz=1048576,hard,timeo=600,retrans=2,noresvport fs-01ab86eb8373f321a.efs.ap-south-1.amazonaws.com:/  
/mnt/efs

[ec2-user@ip-10-0-0-22 ~]$ lsblk
NAME        MAJ:MIN RM SIZE RO TYPE MOUNTPOINTS
nvme0n1     259:0    0 8G  0 disk 
└─nvme0n1p1 259:1    0 8G  0 part /
└─nvme0n1p2 259:2    0 1M  0 part 
└─nvme0n1p3 259:3    0 10M  0 part /boot/efi

[ec2-user@ip-10-0-0-22 ~]$ df -h | grep efs
fs-01ab86eb8373f321a.efs.ap-south-1.amazonaws.com:/ 8.0E  0 8.0E  0% /mnt/efs

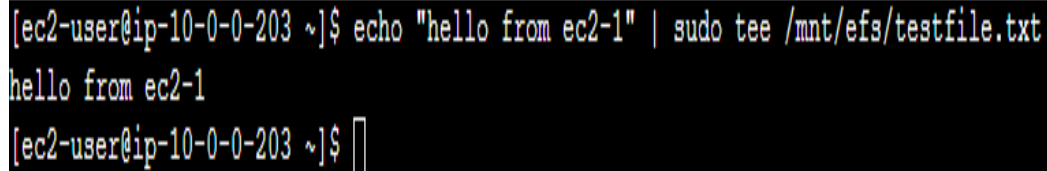
[ec2-user@ip-10-0-0-22 ~]$
```

i-093edeba022f4f7cf (proj-ec2-2)
PublicIPs: 13.203.20.78 PrivateIPs: 10.0.0.22

```
# add to /etc/fstab for persistency (recommended mount helper)
echo "${FS_ID}:/mnt/efs efs _netdev,tls,_netdev 0 0" | sudo tee -a /etc/fstab
```

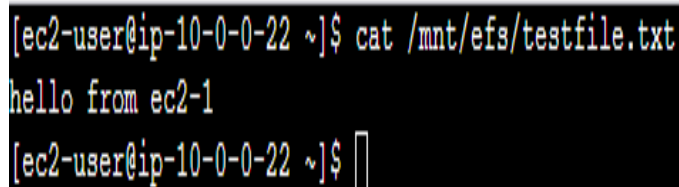
Test sharing:

```
# on EC2-1
echo "hello from ec2-1" | sudo tee /mnt/efs/testfile.txt
```



```
[ec2-user@ip-10-0-0-203 ~]$ echo "hello from ec2-1" | sudo tee /mnt/efs/testfile.txt
hello from ec2-1
[ec2-user@ip-10-0-0-203 ~]$
```

```
# on EC2-2
cat /mnt/efs/testfile.txt # should show the same content
```



```
[ec2-user@ip-10-0-0-22 ~]$ cat /mnt/efs/testfile.txt
hello from ec2-1
[ec2-user@ip-10-0-0-22 ~]$
```

Step 7: CloudWatch Agent Installation and configure CloudWatch Agent (metrics + logs)

We will push system metrics (CPU, memory, disk) and a custom application log file (/var/log/myapp.log) to CloudWatch.

IAM role for EC2

Create an IAM role for EC2 instances with the managed policies:

- CloudWatchAgentServerPolicy (or CloudWatchAgentServerPolicy + AmazonSSMManagedInstanceCore to allow Systems Manager)
- Attach this role to both EC2 instances (or attach at launch time).

aws

Search

[Alt+S]

Global

Account ID: 7719-2394-6960

voclabs/user4088122-harshithagalla00@gmail.com

IAM

Roles

Create role

Step 1

Select trusted entity

Step 2

Add permissions

Step 3

Name, review, and create

Name, review, and create

Role details

Role name

Enter a meaningful name to identify this role.

cloudwatch-iamrole

Maximum 64 characters. Use alphanumeric and "+", "-", "@", ".", "_" characters.

Description

Add a short explanation for this role.

Allows EC2 instances to call AWS services on your behalf.

Maximum 1000 characters. Use letters (A-Z and a-z), numbers (0-9), tabs, new lines, or any of the following characters: "_", "+", "-", "@", "/", "(", ")", "#", "%", "^", ":", ";", "=", "&".

Step 1: Select trusted entities

Edit

Trust policy

```

1 {
2   "Version": "2012-10-17",
3   "Statement": [
4     {
5       "Effect": "Allow",
6       "Principal": {
7         "Service": {
8           "ec2.amazonaws.com"
9         }
10      }
11    ]
12  }
13 }
14
15
16

```

Step 2: Add permissions

Edit

Permissions policy summary

Policy name	Type	Attached as
AmazonSSMManagedInstanceCore	AWS managed	Permissions policy
CloudWatchAgentServerPolicy	AWS managed	Permissions policy

Step 3: Add tags

Add tags - optional

Tags are key-value pairs that you can add to AWS resources to help identify, organize, or search for resources.

No tags associated with the resource.

Add new tag

You can add up to 50 more tags.

Cancel

Previous

Create role

Install agent

Amazon Linux 2

sudo yum install -y amazon-cloudwatch-agent

aws

Search

[Alt+S]

Asia Pacific (Mumbai)

Account ID: 3831-1492-5901

harshitha

CloudShell

Feedback

```

[ec2-user@ip-10-0-0-203 ~]$ sudo yum install -y amazon-cloudwatch-agent
Last metadata expiration check: 0:22:29 ago on Mon Sep 22 14:14:53 2025.
Dependencies resolved.
=====================================================================================================================================
 Package                                Architecture      Version            Repository          Size
=====================================================================================================================================
Installing:
amazon-cloudwatch-agent                x86_64            1.300057.2-1.amzn2023    amazonlinux          135 M
Transaction Summary
=====================================================================================================================================
Install 1 Package
Total download size: 135 M
Installed size: 471 M
Downloading Packages:
amazon-cloudwatch-agent-1.300057.2-1.amzn2023.x86_64.rpm              71 MB/s | 135 MB  00:01
Total
-----
70 MB/s | 135 MB  00:01
Running transaction check
Transaction check succeeded.
Running transaction test
Transaction test succeeded.
Running transaction
Preparing...

```

i-0650425e6977d9c09 (proj-ec2-1)

PublicIPs: 13.204.71.152 PrivateIPs: 10.0.0.203

CloudShell

Feedback

© 2025, Amazon Web Services, Inc. or its affiliates.

Privacy

Terms

Cookie preferences

CloudWatch agent config (sample JSON)

Save as

/opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.json:

sudo nano /opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.json

```
{
  "agent": {
    "metrics_collection_interval": 60,
    "run_as_user": "root"
  },
  "metrics": {
    "namespace": "CWAgent",
    "append_dimensions": {
      "InstanceId": "${aws:InstanceId}"
    },
    "metrics_collected": {
      "cpu": {
        "measurement": ["cpu_usage_active"],
        "metrics_collection_interval": 60
      },
      "mem": {
        "measurement": ["mem_used_percent"],
        "metrics_collection_interval": 60
      },
      "disk": {
        "measurement": ["disk_used_percent"],
        "metrics_collection_interval": 60,
        "resources": ["/data/ebs", "/"]
      }
    }
  },
  "logs": {
    "logs_collected": {
      "files": {
        "collect_list": [
          {
            "file_path": "/var/log/myapp.log",
            "log_group_name": "/proj/myapp",
            "log_stream_name": "${instance_id}"
          }
        ]
      }
    }
  }
}
```


Start the agent

```
sudo /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-ctl \
  -a fetch-config -m ec2 -c
file:/opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.json -s
```

```
[ec2-user@ip-10-0-0-203 ~]$ sudo nano /opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.json
[ec2-user@ip-10-0-0-203 ~]$ sudo /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-ctl \
  -a fetch-config -m ec2 -c file:/opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.json -s
***** processing amazon-cloudwatch-agent *****
I! Trying to detect region from ec2 D! [EC2] Found active network interface I! imds retry client will retry 1 timesSuccessfully fetched the config and
saved in /opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.d/file_amazon-cloudwatch-agent.json.tmp
Start configuration validation...
2025/09/22 14:50:51 Reading json config file path: /opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.d/file_amazon-cloudwatch-agent.json.t
mp ...
2025/09/22 14:50:51 I! Valid Json input schema.
2025/09/22 14:50:51 D! ec2tagger processor required because append_dimensions is set
2025/09/22 14:50:51 Configuration validation first phase succeeded
I! Detecting run_as user...
I! Trying to detect region from ec2
D! [EC2] Found active network interface
I! imds retry client will retry 1 times
/opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent -schematest -config /opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.toml
Configuration validation second phase succeeded
```

Verify agent status:

```
sudo /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-ctl -m ec2 -a status
```

```
[ec2-user@ip-10-0-0-203 ~]$ sudo /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-ctl -m ec2 -a status
{
  "status": "running",
  "starttime": "2025-09-22T14:50:52+00:00",
  "configstatus": "configured",
  "version": "1.300057.2"
}
[ec2-user@ip-10-0-0-203 ~]$
```

Check in AWS Console

- CloudWatch → Logs → log group /proj/myapp and streams.
- CloudWatch → Metrics → CWAgent namespace for metrics.

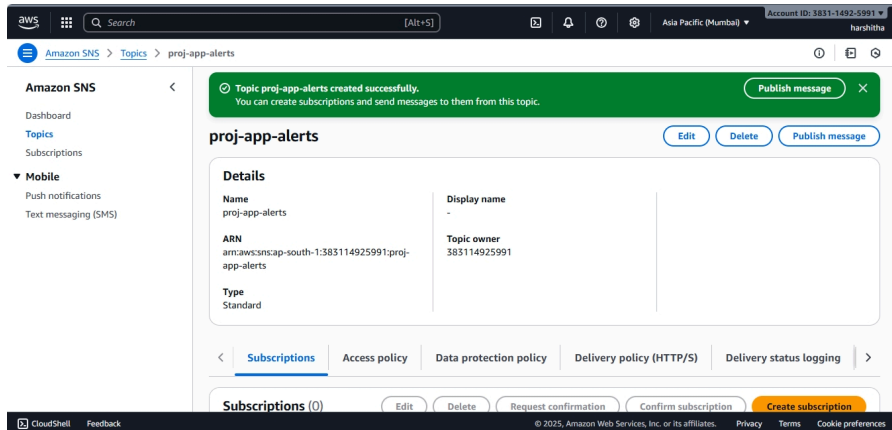
The screenshot shows the AWS CloudWatch Metrics console. The top navigation bar includes the AWS logo, a search bar, and account information (Asia Pacific (Mumbai), Account ID: 3831-1492-5991, user: harshitha). The main content area is titled 'Metrics' and shows a list of metrics for the 'proj-ec2-1' instance. The metrics are displayed in a table with columns for Instance name, Instance ID, Metric name, and Alarms. The table shows one metric: 'cpu_usage_active' with a value of 1.0. The console also includes a search bar for metrics, dimensions, resource IDs, or account IDs, and a 'No alarms' message for the selected metric.

Instance name	Instance ID	Metric name	Alarms
proj-ec2-1	i-0650425e6977d...	cpu_usage_active	No alarms

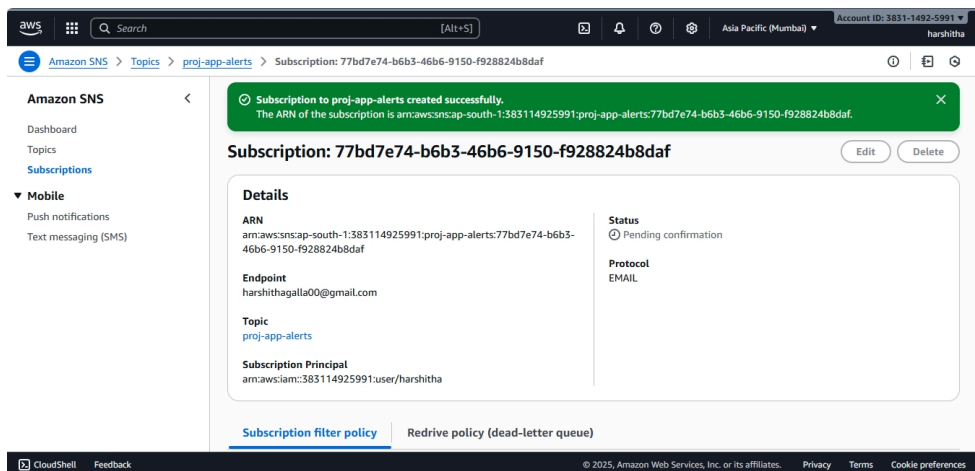
Step 8: Create SNS Topic & Subscription

Console Steps:

1. SNS → Topics → Create topic → Name: proj-app-alerts



2. Topic → Create subscription → Protocol: Email → Endpoint: your email



3. Confirm subscription via email



Subscription confirmed!

You have successfully subscribed.

Your subscription's id is:

`arn:aws:sns:ap-south-1:383114925991:proj-app-alerts:77bd7e74-b6b3-46b6-9150-f928824b8daf`

If it was not your intention to subscribe, [click here to unsubscribe](#).

CLI

create topic

```
TOPIC_ARN=$(aws sns create-topic --name proj-app-alerts --query TopicArn --output text)
```

subscribe your email

```
aws sns subscribe --topic-arn $TOPIC_ARN --protocol email --notification-endpoint  
you@example.com
```

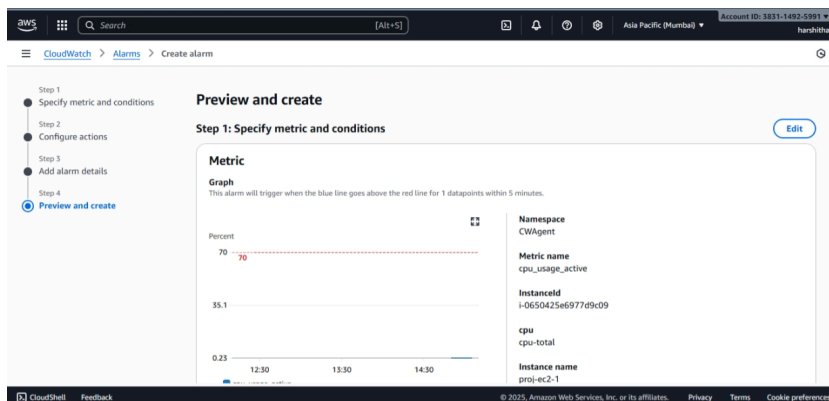
confirm your email via link AWS sends

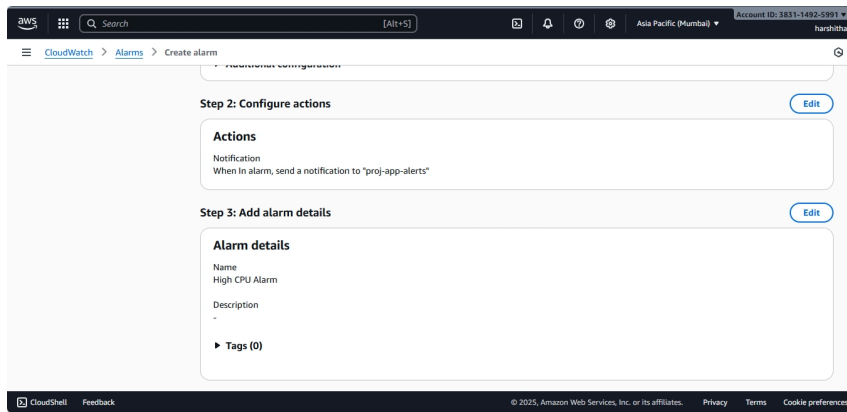
Step 9: Create CloudWatch Alarms that publish to SNS

Alarm 1: High CPU

High CPU Alarm:

- CloudWatch → Alarms → Create Alarm → Select EC2 CPU metric → Condition: >70% → Actions → SNS topic



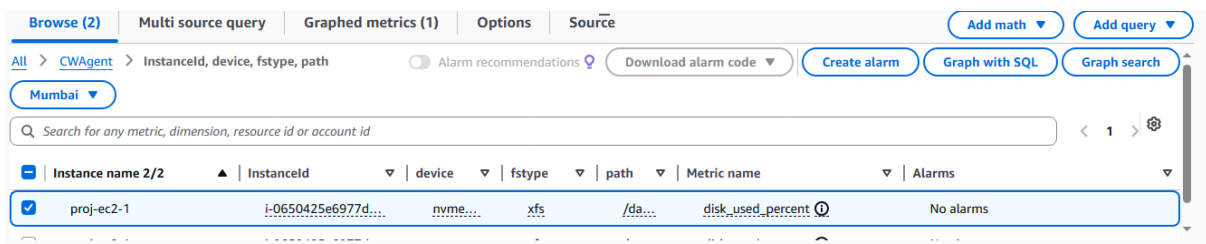


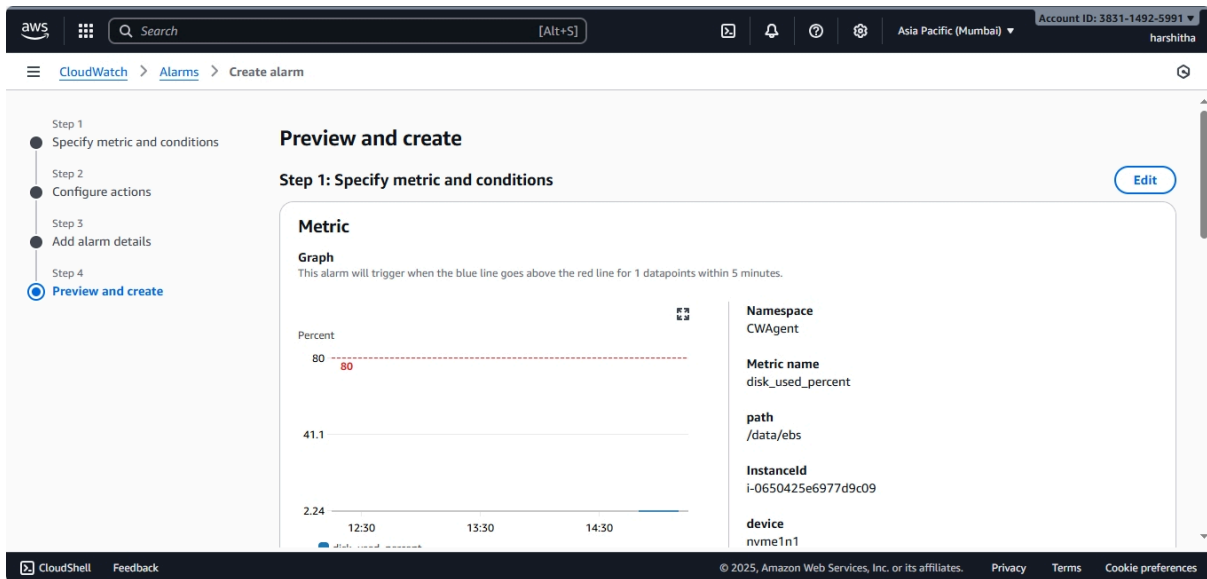
CLI

```
aws cloudwatch put-metric-alarm \
  --alarm-name proj-HighCPU \
  --metric-name CPUUtilization \
  --namespace AWS/EC2 \
  --statistic Average \
  --period 300 --evaluation-periods 1 \
  --threshold 70 \
  --comparison-operator GreaterThanThreshold \
  --dimensions Name=InstanceId,Value=<INSTANCE_ID> \
  --alarm-actions $TOPIC_ARN
```

High Disk Usage Alarm (EBS via CWAgent):

- Metric: CWAgent → disk_used_percent → Path /data/ebs → Condition: >80% → Actions → SNS topic





10 — Application / test scripts

Create a simple test script that writes to EFS and logs to a file on EBS. Put this on both instances.

/home/ec2-user/test-write.sh

```
#!/bin/bash
# write to EFS
echo "$(date) - hello from $(hostname)" >> /mnt/efs/test_shared.txt

# write to local EBS mounted path and log
echo "$(date) - writing to ebs" >> /data/ebs/myapp.log
# also write to app log that CloudWatch collects
echo "$(date) - myapp log from $(hostname)" >> /var/log/myapp.log
```

Make executable and run:

```
chmod +x /home/ec2-user/test-write.sh
./home/ec2-user/test-write.sh
```

Verify:

- /mnt/efs/test_shared.txt content appears on both instances.
- CloudWatch Log: /proj/myapp shows new entries.
- EBS file /data/ebs/myapp.log exists on that instance.

11 — Triggering alarms to validate SNS

CPU alarm test (generate load)

install stress tool

```
sudo yum install -y stress
```

run CPU load for 120s

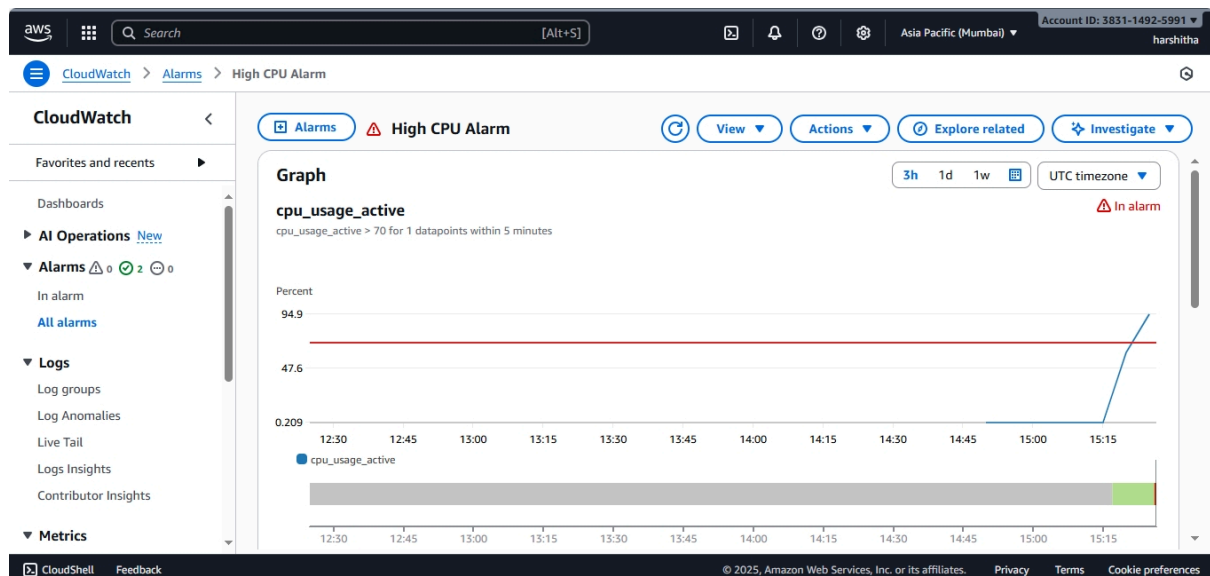
```
stress --cpu 2 --timeout 120
```

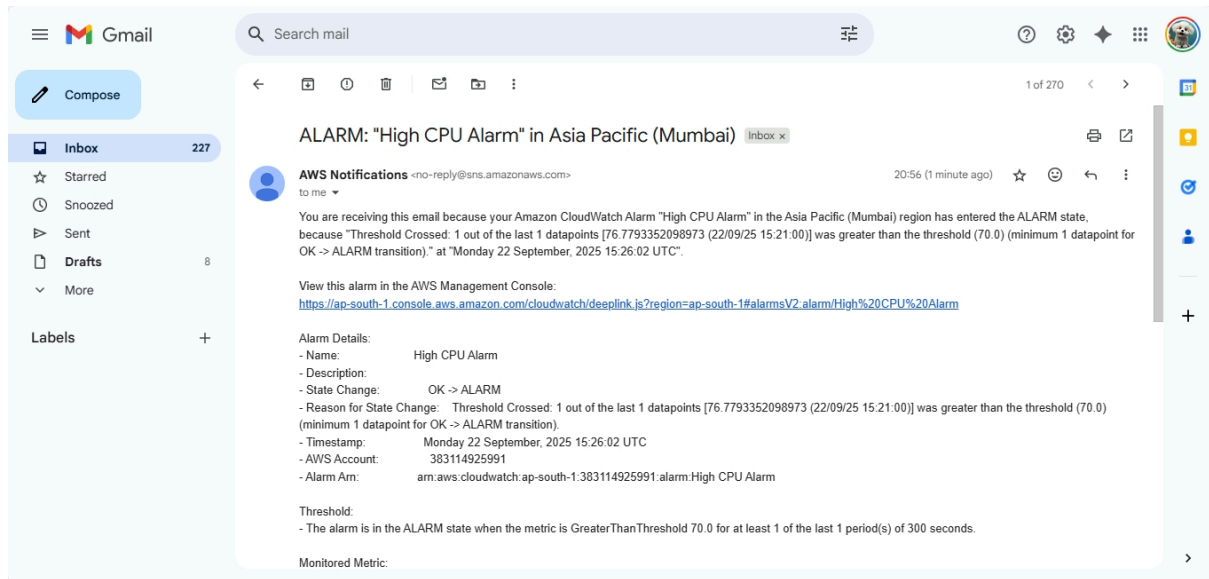
Apply again

If CPU threshold is low enough and period set, alarm should trigger and send email via SNS.

```
Installed:
  stress-1.0.7-2.amzn2023.0.1.x86_64

Complete!
[ec2-user@ip-10-0-0-203 ~]$ stress --cpu 2 --timeout 120
stress: info: [31099] dispatching hogs: 2 cpu, 0 io, 0 vm, 0 hdd
stress: info: [31099] successful run completed in 120s
[ec2-user@ip-10-0-0-203 ~]$ stress --cpu 2 --timeout 120
stress: info: [31225] dispatching hogs: 2 cpu, 0 io, 0 vm, 0 hdd
stress: info: [31225] successful run completed in 120s
```





Disk usage alarm test

Create large file on EBS to reach threshold:

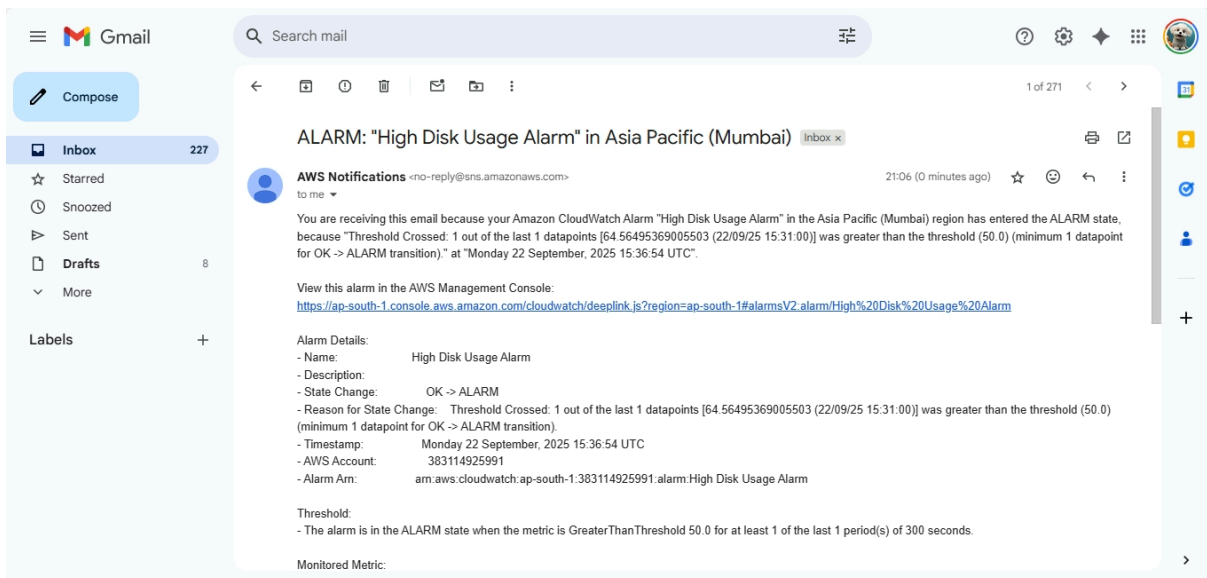
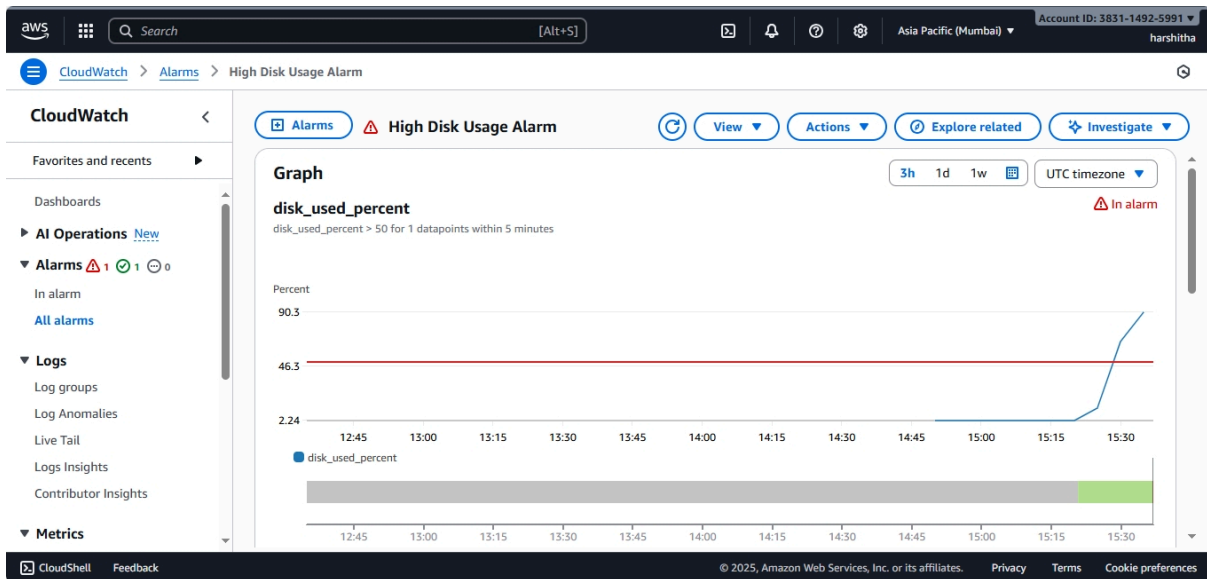
fill EBS with a test file (careful — do not fill root)

`sudo dd if=/dev/zero of=/data/ebs/fillfile bs=1M count=10240 # 10GB`

monitor disk usage

`df -h /data/ebs`

```
[ec2-user@ip-10-0-0-203 ~]$ sudo dd if=/dev/zero of=/data/ebs/fillfile bs=1M count=10240
dd: error writing '/data/ebs/fillfile': No space left on device
9949+0 records in
9948+0 records out
10431234048 bytes (10 GB, 9.7 GiB) copied, 78.4895 s, 133 MB/s
[ec2-user@ip-10-0-0-203 ~]$ sudo dd if=/dev/zero of=/data/ebs/fillfile bs=1M count=10240
dd: error writing '/data/ebs/fillfile': No space left on device
9949+0 records in
9948+0 records out
10431234048 bytes (10 GB, 9.7 GiB) copied, 78.4882 s, 133 MB/s
[ec2-user@ip-10-0-0-203 ~]$ sudo dd if=/dev/zero of=/data/ebs/fillfile bs=1M count=10240
dd: error writing '/data/ebs/fillfile': No space left on device
9949+0 records in
9948+0 records out
10431234048 bytes (10 GB, 9.7 GiB) copied, 78.4891 s, 133 MB/s
[ec2-user@ip-10-0-0-203 ~]$ df -h /data/ebs
Filesystem      Size  Used Avail Use% Mounted on
/dev/nvme1n1    10G   10G   64K 100% /data/ebs
[ec2-user@ip-10-0-0-203 ~]$
```



When percent crosses threshold, CloudWatch alarm fires. Clean up by deleting file.

12 — Tests & validation checklist

- EFS: create file on EC2-1, verify visible on EC2-2.
- EBS: create file, reboot EC2, verify persistence.
- CloudWatch Logs: `/proj/myapp` shows entries from EC2.

- CloudWatch Metrics: see CPU, memory, disk metrics under CWAgent.
- SNS: receive subscription confirmation email.
- CloudWatch Alarm: trigger alarm and receive SNS notification.
- Snapshot: EBS snapshot created and restorable.

13 — Troubleshooting tips

- EFS mount failure: check SGs (NFS 2049), ensure mount targets in each AZ where EC2 resides, install `amazon-efs-utils`.
- EBS not visible: Nitro instances use NVMe names (`/dev/nvme...`), use `lsblk` to find device.
- CloudWatch agent not sending metrics/logs: ensure IAM role attached with `CloudWatchAgentServerPolicy` and agent is running; check agent logs at `/opt/aws/amazon-cloudwatch-agent/logs/amazon-cloudwatch-agent.log`.
- SNS emails not received: confirm subscription from email link.
- Disk alarms: ensure CloudWatch Agent configured to send `disk_used_percent` for the exact mount path.

14 — Cleanup (important to avoid charges)

1. Stop and terminate EC2 instances.
2. Unmount EFS mount points (`sudo umount /mnt/efs`) on instances before termination.
3. Delete EFS file system (ensure it is empty or remove backups).
4. Delete EBS volumes (after detaching) and snapshots.
5. Delete CloudWatch log groups (if you want to avoid logs storage cost):

```
aws logs delete-log-group --log-group-name /proj/myapp
```

6. Delete CloudWatch alarms:

```
aws cloudwatch delete-alarms --alarm-names proj-HighCPU proj-HighDiskUsage
```

7. Delete SNS topic:

```
aws sns delete-topic --topic-arn $TOPIC_ARN
```

8. Delete IAM role/policies created for CloudWatch agent.
9. Delete security groups, subnets, route tables and VPC if you created a new one.